TRACK · SHELL SCRIPTING

SHELL — MODULE 00

The Command Line

Cyber Alliance Piscine — Université Internationale de Casablanca

CYBER ALLIANCE • UIC • BUILD. BREAK. DEFEND.

Université Internationale de Casablanca

UNIVERSITÉ RECONNUE PAR L'ÉTAT

Ground Rules

- This booklet is the only reference for this module — ignore anything you hear second-hand.
- Re-read the statement carefully before submitting; a subject can be corrected between sessions, so re-pull the latest version before your defense.
- Mind the permissions on your files and directories — an unreadable file cannot be graded.
- Every exercise follows the submission procedure described in the last section of this booklet.
- Peer evaluation is combined with an automated grading pass (the "Grader"). The Grader is strict, unforgiving, and not open to negotiation — write clean, defensive code.
- Exercises are ordered from simplest to most advanced. A later exercise is not evaluated if an earlier, simpler one is broken.
- Using a forbidden function or bypassing an explicit constraint is treated as a rule violation and is sanctioned.
- Unless a program is explicitly requested, do not hand in a main() in files meant to be reusable functions.
- Code that does not compile scores zero for that exercise — always compile before you submit.
- Leave nothing in your submission directory beyond what each exercise explicitly asks for.
- Stuck? Ask the person next to you first. Still stuck? Ask a coach. In cybersecurity modules specifically: only ever test techniques against machines and accounts you own or have written authorization to test.

Preamble

Shell00 gets you comfortable at the prompt: variables, exit codes, and the handful of tools (find, wc, sort, mv) that do 80% of a sysadmin's daily work. Every script starts with `#!/bin/bash` and must be made executable before it is turned in. Always quote your variables — an unquoted $var is the single most common bug in this module.

EXERCISE 00 · whoareyou.sh ★■■

Turn in: whoareyou.sh **Allowed:** whoami, hostname, date

Write a script that prints, on three separate lines, the current user, the machine's hostname, and the current date in ISO 8601 format (YYYY-MM-DD).

EXERCISE 01 · count_ext.sh ★★■

Turn in: count_ext.sh **Allowed:** find, wc, grep

Write a script that takes a directory and a file extension as arguments and prints how many files with that extension exist anywhere under that directory (subdirectories included).

Usage: `./count_ext.sh ./myproject .c`

EXERCISE 02 · biggest.sh ★★■

Turn in: biggest.sh **Allowed:** find, du, sort, head

Write a script that takes a directory as an argument and prints the name and size of the single largest file found anywhere under it.

EXERCISE 03 · bulk_rename.sh ★★■

Turn in: bulk_rename.sh **Allowed:** mv, basename

Write a script that takes a directory and a prefix as arguments, and renames every regular file directly inside that directory by prepending the prefix to its name. The script must skip directories and must not touch hidden files.

EXERCISE 04 · path_check.sh ★★■

Turn in: path_check.sh **Allowed:** command -v, echo

Write a script that takes a list of program names as arguments and, for each one, prints ": found" if it exists somewhere in $PATH, or ": missing" otherwise. This is the first building block toward the environment-audit exercises you will meet in the Cybersecurity track.

EXERCISE 05 · arg_loop.sh ★■■

Turn in: arg_loop.sh **Allowed:** none

Write a script that prints every argument it receives, one per line, prefixed with its position ("1: ...", "2: ..."), without using an external command to count them.

Submission Procedure

Work is submitted through the Cyber Alliance grading repository. Create one directory per exercise named exactly as requested in the statement (ex00, ex01, ...), commit only the requested files, and push before the session deadline. Late pushes are not considered.

Shell exercises must be executable (chmod +x) and start with the correct shebang. C exercises must compile cleanly with the flags given on the exercise card, with no warnings and no memory leaks where a checker is mentioned. Cybersecurity exercises that involve a written report must be handed in as a plain text or Markdown file alongside any script requested.